Linked-list

Data Structures & Object-Oriented Programming



Table of Contents

- คำศัพท์พื้นฐาน
- Node, Reference และ Traversal Pattern
- Operation หลักของ Singly Linked List
- คลาส SinglyLinkedList
- Doubly Linked List

ข้อจำกัดของ Dynamic Array ที่เรายังไม่ได้แก้

- คลาส DynamicArray มีจุดเด่นสำคัญคือ การ append ข้อมูลเข้าไปท้ายอาร์เรย์ทำได้ด้วยความซับซ้อนแบบ O(1) โดยเฉลี่ย (amortized) และการเข้าถึงข้อมูลด้วย index ผ่าน __getitem__ ก็ทำได้ในเวลา O(1) เช่นกัน
  - เพราะข้อมูลทั้งหมดถูกจัดเก็บไว้ในหน่วยความจำที่ต่อเนื่องกัน (contiguous memory)
- แต่เมื่อใดก็ตามที่เราต้องการแทรกหรือลบข้อมูลที่ตำแหน่งกลางหรือตำแหน่งต้นอาร์เรย์ (insert_at และ remove_at) เราจำเป็นต้อง เลื่อนข้อมูลที่เหลือทั้งหมด ไปทางซ้ายหรือขวาทีละช่อง เพื่อรักษาความต่อเนื่องของหน่วยความจำเอาไว้ ทำให้ operation เป็น O(n)

สถานการณ์ที่พบได้จริง

- โปรแกรมแก้ไขข้อความ (text editor) ที่มีเคอร์เซอร์: เมื่อผู้ใช้กดปุ่ม Home เพื่อกลับไปต้นบรรทัด แล้วพิมพ์ตัวอักษรใหม่ที่ตำแหน่งนั้นซ้ำหลายครั้ง
  - หากเก็บข้อความไว้ใน Dynamic Array การพิมพ์แต่ละครั้งจะต้องเลื่อนตัวอักษรที่มีอยู่เดิมทั้งหมดไปทางขวาหนึ่งช่อง ซึ่งเป็น O(n) ต่อการพิมพ์หนึ่งครั้ง
- ประวัติการเข้าเว็บของเบราว์เซอร์ (browser history): การกดปุ่ม Back และ Forward คือการเคลื่อนที่ไปมาระหว่างตำแหน่งที่ทราบอยู่แล้ว และการเข้าเว็บใหม่ระหว่างที่ย้อนกลับอยู่คือการแทรก/ลบข้อมูลที่ตำแหน่งกลาง
  - ถ้าใช้ array ก็ต้องเลื่อน elements เช่นเดียวกัน
- เพลย์ลิสต์เพลงที่ต้องการจัดเรียงใหม่ (reorder): การย้ายเพลงจากตำแหน่งหนึ่งไปยังอีกตำแหน่งหนึ่งบ่อยครั้งก็ประสบปัญหาเดียวกัน
- หากใช้ Dynamic Array ต้อง เลื่อน element ทุกครั้งที่มีการแทรกหรือลบที่ไม่ใช่ท้ายอาร์เรย์ เพื่อรักษาความต่อเนื่องของหน่วยความจำ
- Linked List คือโครงสร้างข้อมูลที่ถูกออกแบบมาเพื่อแก้ปัญหานี้โดยตรง โดยไม่ต้องเลื่อน element ใดๆ เลย
  - เพียงแค่เปลี่ยนแปลง “การชี้” ของข้อมูลไม่กี่จุดเท่านั้น

คำศัพท์พื้นฐานที่ต้องเข้าใจ

คำศัพท์พื้นฐานที่ต้องเข้าใจ

- Reference (การอ้างอิง) คือ กลไกที่ตัวแปรใน Python ใช้ในการ “ชี้” ไปยัง object ที่อยู่ในหน่วยความจำ แทนที่จะเก็บค่าของ object นั้นไว้โดยตรง
  - เมื่อเราเขียน a = b ตัวแปร a จะชี้ไปยัง object เดียวกับที่ b ชี้อยู่ ไม่ใช่การคัดลอกข้อมูล
- Node คือหน่วยเก็บข้อมูลพื้นฐานที่สุดของ Linked List
  - เปรียบเสมือนตู้ container หนึ่งใบที่บรรจุข้อมูลของเราไว้ พร้อมกับป้ายบอกทางไปยังตู้ container ใบถัดไป
- Pointer (Python เรียกว่า reference field) คือ ส่วนหนึ่งของ Node ที่ทำหน้าที่ “ชี้” ไปยัง Node อื่น
  - เช่น ฟิลด์ next ที่ชี้ไปยัง Node ถัดไป
- Head คือตัวแปรที่ชี้ไปยัง Node แรกสุดของ list
  - ถ้า list ว่างเปล่า head จะมีค่าเป็น None
- Tail คือตัวแปรที่ชี้ไปยัง Node สุดท้ายของ list
  - การมี tail ช่วยให้เราเข้าถึง Node สุดท้ายได้ทันทีโดยไม่ต้องเดินไล่จากต้น list

คำศัพท์พื้นฐานที่ต้องเข้าใจ (2)

- Traversal คือ กระบวนการเดินไล่ผ่าน Node ทีละตัวตั้งแต่ต้นจนจบ list
  - เป็นรูปแบบการเข้าถึงข้อมูลพื้นฐานที่สุดของ Linked List และจะถูกใช้ซ้ำในแทบทุก operation
- Dangling reference (การอ้างอิงที่ขาดหาย) คือ สถานการณ์ที่เกิดขึ้นเมื่อเราเปลี่ยนค่า pointer ผิดลำดับ จนทำให้ไม่มีตัวแปรใดชี้ไปยัง Node บางตัวอีกต่อไป
  - ส่งผลให้เราไม่สามารถเข้าถึง Node นั้นได้อีก แม้ว่า Node นั้นจะยังไม่ถูกลบออกจากหน่วยความจำก็ตาม
  - (Python จะเก็บกวาดความจำส่วนนี้ให้อัตโนมัติผ่านกลไก garbage collection แต่ข้อมูลที่เราต้องการใช้งานจะสูญหายไปแล้ว)
    - นี่คือสาเหตุของข้อผิดพลาดที่พบบ่อยที่สุดในการเขียน Linked List

จากคลาส DynamicArray สู่แนวคิดของ Linked List

Method ใน `DynamicArray`	พฤติกรรมเดิม	สิ่งที่จะเปลี่ยนไปใน Linked List
`append(value)`	O(1) amortized เพราะข้อมูลต่อท้าย array ที่มีที่ว่างเผื่ออยู่แล้ว	ยังคงเป็น O(1) ได้ ถ้ามี tail pointer แต่กลไกเบื้องหลังคือการเปลี่ยน pointer ไม่ใช่การเขียนค่าลง memory ที่จองไว้
`insert_at(index, value)`	O(n) เพราะต้องเลื่อน element ทุกตัวหลัง index ไปทางขวา	จะกลายเป็น O(1) **ถ้ารู้ตำแหน่ง Node ที่จะแทรกอยู่แล้ว** เพราะไม่ต้องเลื่อนข้อมูลใดๆ เลย เพียงเปลี่ยน pointer
`remove_at(index)`	O(n) เพราะต้องเลื่อน element ทุกตัวมาปิดช่องว่าง	ส่วนของการ “หา” ตำแหน่งยังคงเป็น O(n) แต่ส่วนของการ “ตัดออก” (unlink) เป็น O(1)
`__getitem__(index)`	O(1) เพราะคำนวณตำแหน่งใน memory ได้ทันทีจาก index	กลายเป็น O(n) เพราะไม่มีการคำนวณตำแหน่งได้โดยตรง ต้อง traverse นับจาก head ไปเรื่อยๆ
`contains(value)` / `index_of(value)`	O(n) เดินลูปตรวจสอบทีละตัว	ยังคงเป็น O(n) เหมือนเดิม เพราะหลักการค้นหาแบบ linear search ไม่เปลี่ยน ไม่ว่าจะเก็บข้อมูลแบบใด

- Linked List ไม่ได้ “เร็วกว่า” Dynamic Array ในทุกกรณี แต่มันสลับข้อดีข้อเสียกันคนละจุด
- สิ่งที่ Dynamic Array ทำได้เร็ว (การเข้าถึงด้วย index) Linked List จะทำได้ช้าลง
- สิ่งที่ Dynamic Array ทำได้ช้า (การแทรก/ลบที่ตำแหน่งที่รู้อยู่แล้ว) Linked List จะทำได้เร็วขึ้นอย่างมาก
- การเลือกใช้โครงสร้างข้อมูลที่เหมาะสมจึงขึ้นอยู่กับว่า application ของเราต้องการ operation ชนิดใดบ่อยที่สุด

Node, Reference ใน Python, และ Traversal Pattern

Node : หน่วยเก็บข้อมูลพื้นฐานของ Linked List

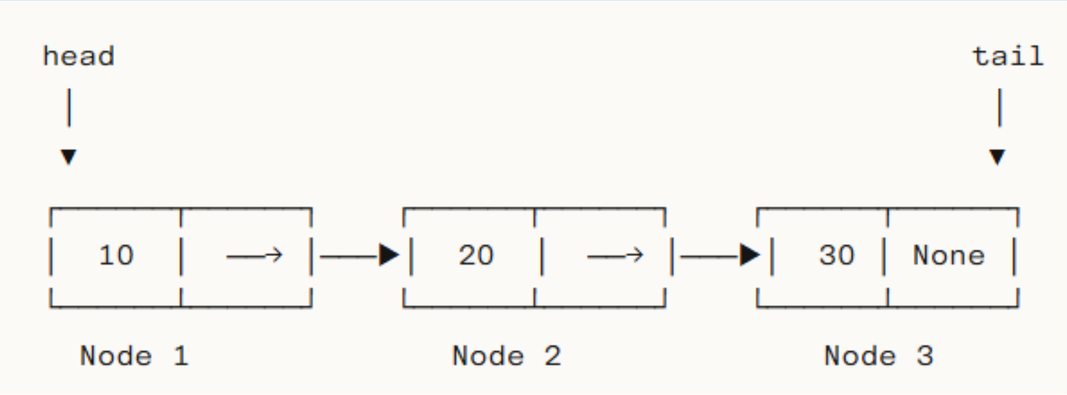
- Linked List ไม่ได้เก็บข้อมูลทั้งหมดไว้ในบล็อกเดียวกัน แต่ละ element จะถูกเก็บไว้ในออบเจกต์แยกกันที่เรียกว่า Node
- โดย Node หนึ่งตัวประกอบด้วยสองส่วนสำคัญเสมอ ได้แก่
  - data คือค่าที่เราต้องการเก็บจริง เช่น ตัวเลข ข้อความ หรือออบเจกต์ใดๆ ก็ได้
  - next คือ reference ที่ชี้ไปยัง Node ถัดไปในลำดับ
    - หากเป็น Node สุดท้ายของ list ค่า next จะเป็น None เพื่อบอกว่าไม่มี Node ใดต่อจากนี้อีกแล้ว
- เมื่อเขียนเป็นโค้ด Python คลาส Node จึงมีหน้าตา ดังนี้

```
class Node:
    """Node หนึ่งตัวใน Singly Linked List"""

    def __init__(self, data):
        self.data = data    # ค่าที่ต้องการเก็บ
        self.next = None    # reference ไปยัง Node ถัดไป (ยังไม่มี จึงเริ่มที่ None)
```

Node : หน่วยเก็บข้อมูลพื้นฐานของ Linked List

- เมื่อสร้าง Node ใหม่ขึ้นมาหนึ่งตัว ค่า next จะถูกตั้งเป็น None เสมอ
  - เพราะ ณ ขณะที่สร้างเสร็จใหม่ๆ Node ตัวนี้ยังไม่ได้เชื่อมต่อกับ Node ใดเลย
  - การเชื่อมต่อจะเกิดขึ้นภายหลังเมื่อเรานำ Node นี้ไปแทรกเข้าไปใน list ผ่าน operation ต่างๆ
- เมื่อนำ Node หลายตัวมาเชื่อมต่อกันด้วยฟิลด์ next เราจะได้ Linked List ที่มีลักษณะดังภาพต่อไปนี้



- ข้อแตกต่างที่สำคัญที่สุดจาก Array คือ Node แต่ละตัวสามารถอยู่ ณ ตำแหน่งใดก็ได้ในหน่วยความจำ ไม่จำเป็นต้องอยู่ติดกัน
- สิ่งที่ทำให้ Node ทั้งหมดกลายเป็น “list” หนึ่งเดียวได้ก็คือสายโซ่ของ reference ที่เชื่อมต่อกันไปเรื่อยๆ
- นี่จึงเป็นเหตุผลว่าทำไมการแทรกหรือลบ Node ตรงกลาง list จึงไม่จำเป็นต้องเลื่อนข้อมูลใดๆ เลย เพียงแค่เปลี่ยนทิศทางของลูกศร

Reference ใน Python

- ใน Python ตัวแปรทุกตัวไม่ได้เก็บ “ค่า” ไว้โดยตรง แต่เก็บ reference ซึ่งเปรียบเสมือนที่อยู่ที่ชี้ไปยัง object ที่แท้จริง ซึ่งอยู่ในหน่วยความจำ
- เมื่อเราเขียนคำสั่ง node.next = another_node
  - ความหมายของคำสั่งนี้คือ “ทำให้ฟิลด์ next ของ node ชี้ไปยัง object เดียวกันกับที่ตัวแปร another_node กำลังชี้อยู่”
  - คำสั่งนี้**ไม่ใช่**การคัดลอกข้อมูลจาก another_node มาใส่ใน node.next แค่ทำให้ทั้งสองฝั่งชี้ไปยัง object เดียวกัน
- ผลที่ตามมาจากหลักการนี้ คือ เมื่อตัวแปร current ชี้อยู่ที่ Node ใดตัวหนึ่ง และเราเขียนคำสั่ง
  - current = current.next
    - คำสั่งนี้**ไม่ได้ย้าย Node ไปไหน** ยังคงอยู่ที่ตำแหน่งเดิมในหน่วยความจำทุกประการ
    - สิ่งที่เกิดขึ้น คือ ตัวแปร current เปลี่ยนไปชี้ที่ Node ถัดไปแทน

Traversal Pattern

- เนื่องจาก Linked List ไม่สามารถคำนวณตำแหน่งของ Node ใดๆ ได้โดยตรงเหมือน Array
  - การจะเข้าถึงหรือประมวลผลข้อมูลภายใน list จึงต้อง “เดิน” ผ่าน Node ทีละตัวตั้งแต่ต้นจนจบเสมอ
  - กระบวนการนี้เรียกว่า traversal
- รูปแบบมาตรฐานของ traversal เขียนเป็นโค้ด Python ได้ดังนี้

```
current = self._head            # เริ่มต้นที่ head เสมอ
while current is not None:      # เดินต่อไปเรื่อยๆ จนกว่าจะถึงปลาย list
    # ประมวลผลกับ current.data ตรงนี้
    current = current.next      # เลื่อนไปยัง Node ถัดไป
```

- **บรรทัดที่ 1 สร้างตัวแปรชั่วคราวชื่อ current แล้วให้ชี้ไปยัง Node แรกของ list ก่อนเสมอ**
  - **ตัวแปร current ทำหน้าที่เป็น “ตัวเดิน” ที่จะเคลื่อนที่ไปตาม list ทีละก้าว โดยที่ self._head ยังคงชี้อยู่ที่ Node แรกตลอดเวลา**
- **บรรทัดที่ 2 คือ เงื่อนไขการหยุด เนื่องจาก Node สุดท้ายของ list จะมีค่า next เป็น None เสมอ การตรวจสอบว่า current is not None จึงเป็นวิธีเดียวที่บอกได้ว่าเรายังไม่ถึงปลาย list**
- **บรรทัดสุดท้าย คือการเลื่อน current ไปยัง Node ถัดไปตามหลักการ reference**

กฎที่ต้องจดจำขึ้นใจ

- ก่อนจะเข้าถึง current.data หรือ current.next ทุกครั้ง จะต้องตรวจสอบก่อนเสมอว่า current is not None
  - หากละเลยขั้นตอนนี้ โปรแกรมจะเกิดข้อผิดพลาดทันทีที่ traversal เดินไปถึงปลาย list ด้วยข้อความ
    - AttributeError: 'NoneType' object has no attribute 'data'
      - ข้อผิดพลาดนี้เกิดขึ้นเพราะเมื่อ current เดินไปถึง Node สุดท้ายแล้วเลื่อนต่ออีกหนึ่งครั้ง ค่าของ current จะกลายเป็น None และ None ไม่มีฟิลด์ data หรือ next ให้เข้าถึง
      - การตรวจสอบเงื่อนไขนี้จึงเป็นข้อบังคับที่ต้องมีอยู่ทุกครั้งที่มีการ traversal

ตัวอย่างการนำ Traversal ไปใช้งานจริง

```python
def count_nodes(head) -> int:
    """นับจำนวน Node ทั้งหมดใน list โดยเริ่มจาก head ที่ได้รับมา"""
    count = 0
    current = head
    while current is not None:
        count += 1
        current = current.next
    return count
```

```python
def print_all_values(head) -> None:
    """พิมพ์ค่า data ของทุก Node เรียงตามลำดับจากต้นจนจบ"""
    current = head
    while current is not None:
        print(current.data)
        current = current.next
```

- ทั้งสองฟังก์ชันนี้ใช้โครงสร้าง traversal แบบเดียวกันทุกประการ แตกต่างกันเพียงแค่สิ่งที่ทำอยู่ภายในลูป (การนับเทียบกับการพิมพ์ค่า)
- ดังนั้น traversal คือ “กุญแจ” สู่ทุก operation

Operation หลักของ Singly Linked List

Singly Linked List คืออะไร

- Linked List ชนิดพื้นฐานที่สุด ประกอบด้วย Node ที่มีฟิลด์ next เพียงทิศทางเดียว
  - แต่ละ Node สามารถ “ชี้ไปข้างหน้า” เท่านั้น ไม่สามารถย้อนกลับไปยัง Node ก่อนหน้าได้
  - การเข้าถึง Node ใดๆ จึงต้องเริ่มต้นจาก head แล้วเดิน (traverse) ไปทีละ Node เสมอ
- โครงสร้างของคลาสที่ใช้แทน Singly Linked List โดยทั่วไปจะประกอบด้วยตัวแปรสามตัว ได้แก่
  - _head ที่ชี้ไปยัง Node แรก
  - _tail ที่ชี้ไปยัง Node สุดท้าย
  - _size ที่เก็บจำนวน Node ทั้งหมดไว้
- การมี _size เพื่อให้การขอทราบจำนวน Node ทั้งหมด (ผ่าน __len__) ทำได้ในเวลา O(1) แทนที่จะต้อง traverse นับใหม่ทุกครั้ง ซึ่งจะกลายเป็น O(n)

Operation: Prepend — เพิ่ม Node ที่ต้น List

- Prepend คือ การเพิ่ม Node ใหม่เข้าไปที่ต้น list โดยให้ Node ใหม่นี้กลายเป็น Node แรกแทนที่ Node เดิม
- จุดเด่นของ operation นี้ คือ สามารถทำได้โดยไม่ต้องแตะต้อง Node อื่นใน list เลย จึงมีความซับซ้อนเพียง O(1)
- พิจารณาตัวอย่าง list ที่มีอยู่แล้วดังนี้
  - Before: head → [10] → [20] → [30] → None
- หากต้องการเพิ่มค่า 5 เข้าไปที่ต้น list การดำเนินการต้องทำเป็น 2 ขั้นตอนตามลำดับที่แน่นอน
  - **ขั้นที่ 1:** ให้ Node ใหม่ชี้ไปยัง Node ที่เคยเป็น head เดิมก่อน
    - new_node.next = self._head
    - [5] ————————→ [10] → [20] → [30] → None
  - **ขั้นที่ 2:** จากนั้นจึงเปลี่ยนให้ _head มาชี้ที่ Node ใหม่แทน
    - self._head = new_node
    - head → [5] → [10] → [20] → [30] → None

Operation: Append ด้วย Tail Pointer

- Append คือ การเพิ่ม Node ใหม่เข้าไปที่ท้าย list
- หากไม่มีการเก็บ _tail ไว้ การ append จะต้อง traverse จาก _head ไปจนถึง Node สุดท้ายก่อนทุกครั้ง ซึ่งมีความซับซ้อนเป็น O(n)
  - แต่เมื่อมี _tail ที่ชี้ไปยัง Node สุดท้ายอยู่เสมอ การ append จะทำได้ในเวลา O(1)
- พิจารณา list เดิมที่มี _tail ชี้อยู่ที่ Node ค่า 30

```
Before:  head → [10] → [20] → [30] → None
                                 ↑
                                tail
```

- หากต้องการเพิ่มค่า 40 เข้าไปที่ท้าย list
  - ขั้นที่ 1: ให้ Node ที่ _tail ชี้อยู่ เชื่อมต่อไปยัง Node ใหม่
    - self._tail.next = new_node

```
head → [10] → [20] → [30] → [40] → None
                       ↑
                      tail (ยังคงชี้อยู่ที่ 30 เหมือนเดิม)
```

  - ขั้นที่ 2: จากนั้นจึงเปลี่ยนให้ _tail มาชี้ที่ Node ใหม่แทน
    - self._tail = new_node

```
head → [10] → [20] → [30] → [40] → None
                             ↑
                            tail
```

กรณีพิเศษ: หาก list ว่างเปล่าอยู่ก่อนการ append (_tail เป็น None) จะไม่สามารถเขียน self._tail.next = new_node ได้ เพราะ _tail ยังไม่มี Node ใดให้ชี้อยู่ ในกรณีนี้จึงต้องตั้งค่าทั้ง _head และ _tail ให้ชี้ไปยัง Node ใหม่โดยตรง

Operation: Insert After Known Node

- Operation นี้คือเหตุผลหลักที่ทำให้ Linked List มีข้อได้เปรียบเหนือ Dynamic Array อย่างชัดเจน
- หากเรารู้อยู่แล้วว่าต้องการแทรก Node ใหม่ไว้หลัง Node ใด (สมมติเรียก Node นั้นว่า current) การแทรกทั้งหมดสามารถทำได้ด้วยการเปลี่ยนแปลง pointer เพียงสองตัวเท่านั้น โดยมีความซับซ้อนเป็น O(1)
- พิจารณา list head → [10] → [20] → [30] → None
  - ต้องการแทรกค่า 25 ไว้หลัง Node ที่ current ชี้อยู่ (คือ Node ค่า 20)
- **ขั้นที่ 1** : ให้ Node ใหม่ชี้ไป Node ที่อยู่ถัดจาก current เดิม เพื่อ “บันทึก” ตำแหน่งของส่วนที่เหลือของ list เอาไว้
  - new_node.next = current.next
  - [25] ——————————————→ [30] → None
- **ขั้นที่ 2:** จากนั้นจึงให้ current ชี้ไปยัง Node ใหม่แทน
  - current.next = new_node
  - head → [10] → [20] → [25] → [30] → None

กฎทั่วไปที่ใช้ได้กับการแทรก Node ทุกกรณี ให้ตั้งค่า pointer ของ Node ใหม่ก่อนเสมอ เพื่อบันทึกการเชื่อมต่อกับส่วนที่เหลือของ list ไว้ก่อน จากนั้นจึงค่อยเปลี่ยน pointer ของ Node เดิมให้มาชี้ที่ Node ใหม่

Operation: Delete Node

- การลบ Node ออกจาก Singly Linked List
  - ให้ Node ที่อยู่ ก่อนหน้า Node ที่ต้องการลบ (เรียกว่า prev) ข้ามการอ้างอิงไปยัง Node ที่อยู่ถัดจาก Node ที่ต้องการลบโดยตรง
- พิจารณา list head → [10] → [20] → [30] → None
  - ต้องการลบ Node ค่า 20 ออก และสมมติว่า prev ชี้อยู่ที่ Node ค่า 10 อยู่แล้ว
  - prev.next = prev.next.next
  - head → [10] → [30] → None
  - [20] ไม่มีตัวแปรใดชี้ไปหาอีกต่อไป → Python จัดการเก็บกวาดความจำให้อัตโนมัติ)
- คำสั่งเพียงบรรทัดเดียวนี้ทำให้ Node ค่า 20 หลุดออกจากสายโซ่ของ list ไปทันที
- operation นี้แบ่งได้เป็นสองส่วน
  - ส่วนแรกคือการ **หา** prev ซึ่งต้อง traverse จาก _head มาเรื่อยๆ จึงมีความซับซ้อน O(n)
  - ส่วนที่สองคือการ **ตัดการเชื่อมต่อ** (unlink) ซึ่งเป็นเพียงการเปลี่ยน pointer หนึ่งตัวเท่านั้น จึงเป็น O(1)
  - เมื่อรวมสองส่วน ความซับซ้อนโดยรวม จึงเป็น **O(n)**

การลบ head

- เมื่อ Node ที่ต้องการลบคือ Node แรกของ list (_head เอง) จะไม่มี prev ให้ใช้งาน (เพราะไม่มี Node ใดอยู่ก่อนหน้า head) จึงต้องอัปเดต _head โดยตรงแทน
  - self._head = self._head.next
- หาก list มี Node อยู่เพียงตัวเดียวก่อนการลบ หลังจากลบแล้ว _head จะกลายเป็น None ในกรณีนี้จะต้องอัปเดต _tail = None ด้วย
  - เพราะ list ได้กลายเป็น list ว่างเปล่าไปแล้ว
  - หากลืมขั้นตอนนี้ _tail จะยังคงชี้ไปยัง Node ที่ถูกลบไปแล้ว ซึ่งจะทำให้การ append ครั้งถัดไปทำงานผิดพลาด

การ append บน list ที่ว่างเปล่า

- เมื่อ _tail เป็น None จะไม่สามารถเขียน self._tail.next = new_node ได้
  - ต้องตรวจสอบเงื่อนไขนี้ก่อนเสมอ
- ตั้งค่าทั้ง _head และ _tail ให้ชี้ไปยัง Node ใหม่โดยตรงหาก list ว่างเปล่าอยู่ก่อนหน้า
- ทั้งสองกรณีนี้มีหลักคิดร่วมกันคือ ทุกครั้งที่ operation ใดๆ อาจส่งผลกระทบต่อ _head หรือ _tail (ไม่ว่าจะเป็นเพราะ list กำลังจะว่างเปล่า หรือกำลังจะมี Node แรกเกิดขึ้น)
  - ต้องตรวจสอบและอัปเดตค่าเหล่านี้อย่างระมัดระวังเป็นพิเศษ
    - เพราะ _head และ _tail คือจุดเริ่มต้นของทุก operation ในอนาคต
  - หากค่าทั้งสองนี้ผิดพลาด operation ถัดไปทั้งหมดจะผิดพลาดตามไปด้วย

คลาส SinglyLinkedList

คลาส Node และ SinglyLinkedList

```python
class Node:
    """Node หนึ่งตัวใน Singly Linked List"""

    def __init__(self, data):
        self.data = data
        self.next = None
```

```python
class SinglyLinkedList:
    """
    Singly Linked List ที่มีทั้ง head และ tail pointer
    การมี tail ทำให้ append เป็น O(1) แทนที่จะเป็น O(n)
    """

    def __init__(self):
        self._head = None
        self._tail = None
        self._size = 0

    def __len__(self) -> int:
        return self._size
```

__init__ ตั้งค่าเริ่มต้นให้

_head, _tail เป็น None

_size เป็น 0 เสมอ

เพราะ list ที่เพิ่งถูกสร้างขึ้นย่อมว่างเปล่า

คลาส Node และ SinglyLinkedList (2)

```
def __str__(self) -> str:
    result = []
    current = self._head
    while current is not None:
        result.append(str(current.data))
        current = current.next
    if not result:
        return "None"
    return " → ".join(result) + " → None"
```

__len__ และ __str__ เป็น dunder method

__str__ ของ Linked List

- ต้องใช้ traversal pattern ในการไล่เก็บค่าจากทุก Node ก่อนนำมาต่อกันเป็น string
- ไม่สามารถเข้าถึงข้อมูลทั้งหมดได้ในครั้งเดียวเหมือน array

คลาส SinglyLinkedList (3)

```
def prepend(self, data) -> None:
    """เพิ่ม Node ที่ต้น list — O(1)"""
    new_node = Node(data)
    new_node.next = self._head          # ขั้นที่ 1: ชี้ไปยัง head เดิมก่อน
    self._head = new_node               # ขั้นที่ 2: อัปเดต head
    if self._tail is None:              # list เคยว่างเปล่ามาก่อน
        self._tail = new_node
    self._size += 1
```

- เงื่อนไข if self._tail is None ซึ่งเป็นการตรวจสอบว่า list เคยว่างเปล่ามาก่อนหรือไม่
- หากว่างเปล่า Node ที่เพิ่งเพิ่มเข้าไปนี้จะเป็นทั้ง head และ tail ในเวลาเดียวกัน

คลาส SinglyLinkedList (4)

```
def append(self, data) -> None:
    """เพิ่ม Node ที่ท้าย list — O(1) ด้วย tail pointer"""
    new_node = Node(data)
    if self._tail is None:              # list ว่างเปล่าอยู่ก่อนหน้า
        self._head = new_node
        self._tail = new_node
    else:
        self._tail.next = new_node      # ขั้นที่ 1: ต่อจาก tail เดิม
        self._tail = new_node           # ขั้นที่ 2: อัปเดต tail
    self._size += 1
```

append แยกเป็นสองกรณีอย่างชัดเจน คือ

- กรณี list ว่างเปล่า (ต้องตั้งทั้ง _head และ _tail)
- กรณี list มีข้อมูลอยู่แล้ว (ใช้ _tail ในการต่อ Node ใหม่โดยไม่ต้อง traverse)

คลาส SinglyLinkedList (5)

```
def search(self, target) -> bool:
    """ค้นหาว่ามีค่า target อยู่ใน list หรือไม่ — O(n)"""
    current = self._head
    while current is not None:
        if current.data == target:
            return True
        current = current.next
    return False
```



คลาส SinglyLinkedList (6)

```
def insert_after(self, target, data) -> bool:
    """
    แทรก Node ใหม่ (data) ไว้หลัง Node แรกที่มีค่าเท่ากับ target
    คืนค่า True หากพบและแทรกสำเร็จ, False หากไม่พบ target ใน list
    """
    current = self._head
    while current is not None:
        if current.data == target:
            new_node = Node(data)
            new_node.next = current.next # ขั้นที่ 1: บันทึกส่วนที่เหลือของ list
            current.next = new_node      # ขั้นที่ 2:เชื่อม current เข้ากับ node ใหม่
            if current is self._tail:    # target คือ tail เดิม
                self._tail = new_node
            self._size += 1
            return True
        current = current.next
    return False
```

- ต้อง traverse เพื่อค้นหา target ก่อน (ส่วนนี้เป็น O(n))
- เมื่อพบแล้วจึงทำการแทรกตามลำดับสองขั้นตอน (ส่วนนี้เป็น O(1))
- การตรวจสอบ if current is self._tail
  - ถ้า target ที่พบเป็น Node สุดท้ายของ list พอดี การแทรก Node ใหม่หลังจากนั้นจะทำให้ Node ใหม่กลายเป็น tail ตัวใหม่
  - หากลืมอัปเดตจุดนี้ _tail จะยังคงชี้ผิดตำแหน่งและส่งผลต่อการ append ครั้งถัดไป

คลาส SinglyLinkedList (7)

```
def delete(self, target) -> bool:
    """
    ลบ Node แรกที่มีค่าเท่ากับ target ออกจาก list
    คืนค่า True หากลบสำเร็จ, False หากไม่พบ target ใน list
    """
    if self._head is None:
        return False

    if self._head.data == target:              # กรณีพิเศษ: ลบที่ head
        self._head = self._head.next
        if self._head is None:                  # list เหลือ node เดียวก่อนลบ
            self._tail = None
        self._size -= 1
        return True

    prev = self._head
    current = self._head.next
    while current is not None:
        if current.data == target:
            prev.next = current.next
            if current is self._tail:           # กรณีพิเศษ: ลบที่ tail
                self._tail = prev
            self._size -= 1
            return True
        prev = current
        current = current.next
    return False
```

ต้องแยกพิจารณา 3 กรณี

- (1) การลบที่ head ซึ่งไม่มี prev ให้ใช้งาน จึงอัปเดต _head โดยตรง
- (2) กรณีที่ list เหลือ Node เดียวก่อนลบ ซึ่งต้องอัปเดต _tail = None ควบคู่ไปด้วย
- (3) การลบที่ tail ซึ่งต้องอัปเดต _tail ให้ย้อนกลับไปชี้ที่ prev แทน

ทดสอบการทำงาน

```
lst = SinglyLinkedList()
print(lst)
print(len(lst))

lst.append(20)
lst.append(30)
lst.prepend(10)
print(lst)
print(len(lst))

lst.append(40)
lst.prepend(5)
print(lst)

print(lst.search(20))
print(lst.search(99))

lst.insert_after(20, 25)
print(lst)

lst.delete(5)
print(lst)
print(len(lst))

lst.delete(40)
print(lst)

lst.append(50)
print(lst)
```

```
None
0

10 → 20 → 30 → None
3

5 → 10 → 20 → 30 → 40 → None

True
False

5 → 10 → 20 → 25 → 30 → 40 → None

10 → 20 → 25 → 30 → 40 → None
5

10 → 20 → 25 → 30 → None

10 → 20 → 25 → 30 → 50 → None
```

Doubly Linked List

Doubly Linked List คืออะไร

- Singly Linked List มีข้อจำกัดสำคัญอยู่ 1 อย่าง คือ Node แต่ละตัวสามารถ “มองไปข้างหน้า” ได้เพียงทิศทางเดียวผ่านฟิลด์ next เท่านั้น
- หากเราอยู่ที่ Node ใดตัวหนึ่งแล้วต้องการทราบว่า Node ก่อนหน้าคือตัวไหน จะไม่มีทางทราบได้เลยนอกจาก traverse ใหม่จาก _head มาเรื่อยๆ
  - ไม่มีประสิทธิภาพอย่างยิ่ง โดยเฉพาะเมื่อต้องการลบ Node ใดตัวหนึ่ง (ต้อง traverse หา prev ก่อนเสมอ)
- Doubly Linked List แก้ปัญหานี้โดยเพิ่มฟิลด์ที่สามเข้าไปใน Node นอกเหนือจาก data และ next นั่นคือฟิลด์ prev ซึ่งชี้ย้อนกลับไปยัง Node ก่อนหน้าโดยตรง
  - Node ทุกตัวรู้จักทั้งเพื่อนบ้านฝั่งซ้ายและฝั่งขวาของตัวเองพร้อมกัน

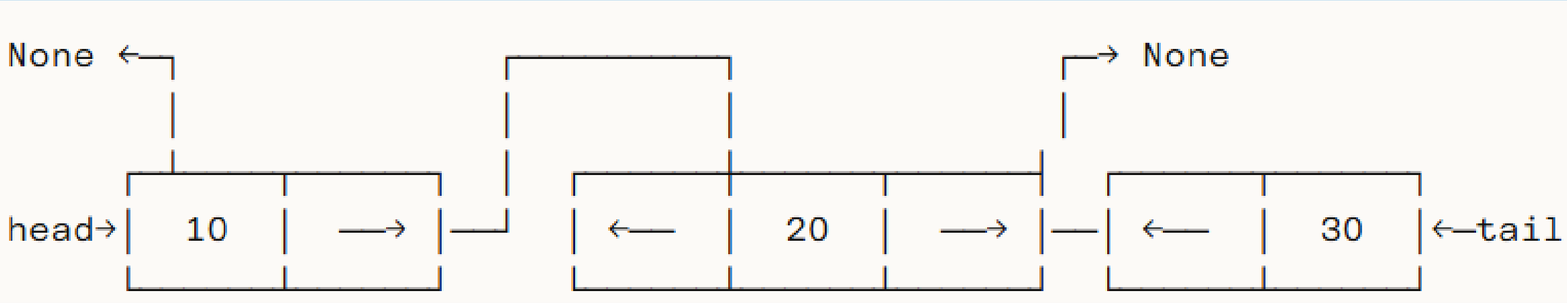
Doubly Linked List คืออะไร

คลาส DoubleNode มีหน้าตาดังนี้

```
class DoubleNode:
    """Node หนึ่งตัวใน Doubly Linked List"""

    def __init__(self, data):
        self.data = data
        self.next = None    # ชี้ไปยัง Node ถัดไป
        self.prev = None    # ชี้ไปยัง Node ก่อนหน้า
```

เมื่อนำ DoubleNode หลายตัวมาเชื่อมต่อกัน จะได้โครงสร้างที่มีลูกศรวิ่งอยู่สองทิศทางระหว่างทุกคู่ Node ดังภาพต่อไปนี้



Doubly Linked List

การมีฟิลด์ prev เพิ่มขึ้นมานี้ทำให้ Doubly Linked List มีข้อได้เปรียบกว่า Singly Linked List ดังนี้

1. สามารถ traverse ย้อนจากท้ายมาต้น list ได้
2. การลบ Node ที่ตำแหน่งใดก็ตาม สามารถทำได้ในเวลา O(1) หากมี reference ไปยัง Node นั้นอยู่แล้ว เพราะไม่ต้องเสียเวลา traverse หา prev แยกต่างหากอีกต่อไป

แต่ข้อได้เปรียบเหล่านี้มาพร้อมกับต้นทุนที่ต้องแลก

- Node แต่ละตัวใช้หน่วยความจำมากขึ้น เนื่องจากต้องเก็บ pointer เพิ่มอีกหนึ่งตัว
- ทุก operation ที่แก้ไข list จะต้องดูแลรักษาทั้ง next และ prev ให้ถูกต้องสอดคล้องกันเสมอ
    - จำนวนจุดที่ต้องระวังความผิดพลาดก็เพิ่มขึ้นเป็นสองเท่าเมื่อเทียบกับ Singly Linked List

Operation: Prepend ของ Doubly Linked List

Prepend — เพิ่ม Node ใหม่ที่ต้น list

หลักการใกล้เคียงกับ Singly Linked List แต่ต้องเพิ่มการอัปเดตฟิลด์ prev เข้าไปด้วยทุกครั้ง

```
def prepend(self, data) -> None:
    new_node = DoubleNode(data)
    if self._head is None:                  # list ว่างเปล่าอยู่ก่อนหน้า
        self._head = new_node
        self._tail = new_node
    else:
        new_node.next = self._head          # ขั้นที่ 1: ชี้ไปยัง head เดิม
        self._head.prev = new_node          # ขั้นที่ 2: head เดิมชี้ย้อนกลับมาที่ node ใหม่
        self._head = new_node               # ขั้นที่ 3: อัปเดต head
    self._size += 1
```

- สังเกตว่ามีขั้นตอนเพิ่มขึ้นมา 1 ขั้นจากเวอร์ชัน Singly นั่นคือ self._head.prev = new_node ซึ่งทำหน้าที่ “ชี้ย้อน” ของ Node ที่เคยเป็น head เดิมให้กลับมาชี้ที่ Node ใหม่
- หากลืมขั้นตอนนี้ ฟิลด์ prev ของ Node เดิมจะยังคงเป็น None ทั้งที่จริงๆ แล้วตอนนี้มี Node อยู่ก่อนหน้ามันแล้ว ส่งผลให้การ traverse ย้อนกลับ ให้ผลลัพธ์ผิดพลาดทันที

Operation: Append ของ Doubly Linked List

```
def append(self, data) -> None:
    new_node = DoubleNode(data)
    if self._tail is None:                  # list ว่างเปล่าอยู่ก่อนหน้า
        self._head = new_node
        self._tail = new_node
    else:
        new_node.prev = self._tail          # ขั้นที่ 1: node ใหม่ชี้ย้อนกลับไปที่ tail เดิม
        self._tail.next = new_node          # ขั้นที่ 2: tail เดิมชี้ไปข้างหน้าที่ node ใหม่
        self._tail = new_node               # ขั้นที่ 3: อัปเดต tail
    self._size += 1
```

ต้องอัปเดต pointer สองทิศทางแทนที่จะเป็นทิศทางเดียว

Operation: Insert After

- การแทรก Node ใหม่ไว้หลัง Node ที่รู้อยู่แล้ว (current) มีความซับซ้อนมากกว่าใน Singly Linked List
- ต้องดูแลทั้งฟิลด์ next และ prev ของ Node ที่เกี่ยวข้องทั้งหมด ทำให้โดยรวมแล้วต้องอัปเดต pointer ถึง 4 จุด

พิจารณา list head → [10] ⇄ [20] ⇄ [30] → None โดยต้องการแทรกค่า 25 ไว้หลัง current (Node ค่า 20)

```
def insert_after(self, target, data) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            new_node = DoubleNode(data)
            new_node.next = current.next          # จุดที่ 1
            new_node.prev = current               # จุดที่ 2
            if current.next is not None:
                current.next.prev = new_node      # จุดที่ 3 (กรณี current ไม่ใช่ tail)
            else:
                self._tail = new_node             # current เดิมคือ tail
            current.next = new_node               # จุดที่ 4
            self._size += 1
            return True
        current = current.next
    return False
```

- ต้องตั้งค่า pointer ของ Node ใหม่ก่อนเสมอ (จุดที่ 1 และ 2) เพื่อบันทึกการเชื่อมต่อกับ Node ข้างเคียงทั้งสองฝั่งไว้ให้ครบก่อน
- จากนั้นจึงค่อยเปลี่ยน pointer ของ Node เดิม (จุดที่ 3 และ 4)

จุดที่ 3 มีเงื่อนไข if current.next is not None เพราะหาก current ที่พบเป็น Node สุดท้ายของ list พอดี จะไม่มี Node ใดให้ทำการอัปเดตฟิลด์ prev ได้
กรณีนี้สิ่งที่ต้องทำคือปรับ _tail ให้ชี้ไปยัง Node ใหม่แทน เนื่องจาก Node ใหม่นี้ได้กลายเป็น Node สุดท้ายของ list แล้ว

Operation: Delete

- สำหรับ Doubly Linked List ทุก Node รู้จัก prev ของตัวเองตลอดเวลา ดังนั้นเมื่อพบ Node ที่ต้องการลบ (เรียกว่า current) การตัดออกจากสายโซ่จึงทำได้ทันทีโดยไม่ต้อง traverse หา prev แยกต่างหากอีก
  - นี่คือเหตุผลที่การลบ Node ที่ตำแหน่งใดก็ได้ใน Doubly Linked List เป็น O(1) เมื่อมี reference ไปยัง Node นั้นอยู่แล้ว ต่างจาก Singly Linked List ที่เป็น O(n)

```python
def delete(self, target) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            if current.prev is not None:
                current.prev.next = current.next      # ไม่ใช่ head: เชื่อม prev ข้ามไป
            else:
                self._head = current.next             # เป็น head: อัปเดต head โดยตรง

            if current.next is not None:
                current.next.prev = current.prev  # ไม่ใช่ tail: เชื่อม next ข้ามกลับมา
            else:
                self._tail = current.prev             # เป็น tail: อัปเดต tail โดยตรง

            self._size -= 1
            return True
        current = current.next
    return False
```

- method นี้ยังคงต้อง traverse เพื่อ ค้นหา ตำแหน่งของ target อยู่ (ผ่านการเปรียบเทียบค่า data) จึงยังมีความซับซ้อนโดยรวมเป็น O(n)

- ความแตกต่างที่แท้จริงจะปรากฏชัดในสถานการณ์ที่เรามี reference ไปยัง Node นั้นอยู่แล้ว โดยไม่ต้องค้นหา

Operation: Delete

```python
def delete(self, target) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            if current.prev is not None:
                current.prev.next = current.next      # ไม่ใช่ head: เชื่อม prev ข้ามไป
            else:
                self._head = current.next             # เป็น head: อัปเดต head โดยตรง

            if current.next is not None:
                current.next.prev = current.prev  # ไม่ใช่ tail: เชื่อม next ข้ามกลับมา
            else:
                self._tail = current.prev             # เป็น tail: อัปเดต tail โดยตรง

            self._size -= 1
            return True
        current = current.next
    return False
```

code นี้ จัดการทั้งสี่กรณีที่เป็นไปได้ครบถ้วน

- ลบ Node ที่เป็นทั้ง head และ tail พร้อมกัน (list เหลือ Node เดียว)
- ลบที่ head แต่ไม่ใช่ tail
- ลบที่ tail แต่ไม่ใช่ head
- ลบ Node ตรงกลางที่ไม่ใช่ทั้งสองอย่าง

โดยไม่จำเป็นต้องเขียนแยกเป็นกรณีพิเศษเหมือนใน Singly Linked List เพราะเงื่อนไข if current.prev is not None และ if current.next is not None ครอบคลุมทุกกรณีไว้แล้ว

Backward Traversal

- เนื่องจาก Doubly Linked List มี _tail และฟิลด์ prev ในทุก Node เราจึงสามารถ traverse จากปลาย list ย้อนกลับมาต้น list ได้
  - เป็นความสามารถที่ Singly Linked List ไม่มี
- backward traversal มีโครงสร้างคล้ายกับ traversal ปกติทุกประการ เพียงแต่สลับจุดเริ่มต้นและทิศทางการเดิน

```
def traverse_backward(self) -> list:
    """คืนค่า list ของ data ทุกตัว เรียงจากท้ายมาต้น"""
    result = []
    current = self._tail            # เริ่มจาก tail แทนที่จะเป็น head
    while current is not None:
        result.append(current.data)
        current = current.prev      # เดินถอยหลังผ่าน prev แทนที่จะเป็น next
    return result
```

ความสามารถนี้มีประโยชน์อย่างมากในสถานการณ์จริงหลายอย่าง เช่น การกดปุ่ม Back ของเบราว์เซอร์ที่ต้องย้อนกลับไปยังหน้าเว็บก่อนหน้า หรือฟังก์ชัน undo ในโปรแกรมแก้ไขข้อความ

คลาสสมบูรณ์: DoubleNode และ DoublyLinkedList

```python
class DoubleNode:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None
```

```python
class DoublyLinkedList:
    def __init__(self):
        self._head = None
        self._tail = None
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def __str__(self) -> str:
        result = []
        current = self._head
        while current is not None:
            result.append(str(current.data))
            current = current.next
        if not result:
            return "None"
        return " ⇄ ".join(result) + " ⇄ None"
```

คลาสสมบูรณ์: DoubleNode และ DoublyLinkedList

```
def prepend(self, data) -> None:
    new_node = DoubleNode(data)
    if self._head is None:
        self._head = new_node
        self._tail = new_node
    else:
        new_node.next = self._head
        self._head.prev = new_node
        self._head = new_node
    self._size += 1
```

```
def append(self, data) -> None:
    new_node = DoubleNode(data)
    if self._tail is None:
        self._head = new_node
        self._tail = new_node
    else:
        new_node.prev = self._tail
        self._tail.next = new_node
        self._tail = new_node
    self._size += 1
```

คลาสสมบูรณ์: DoubleNode และ DoublyLinkedList

```python
def search(self, target) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            return True
        current = current.next
    return False
```

```python
def insert_after(self, target, data) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            new_node = DoubleNode(data)
            new_node.next = current.next
            new_node.prev = current
            if current.next is not None:
                current.next.prev = new_node
            else:
                self._tail = new_node
            current.next = new_node
            self._size += 1
            return True
        current = current.next
    return False
```

คลาสสมบูรณ์: DoubleNode และ DoublyLinkedList

```
def delete(self, target) -> bool:
    current = self._head
    while current is not None:
        if current.data == target:
            if current.prev is not None:
                current.prev.next = current.next
            else:
                self._head = current.next
            if current.next is not None:
                current.next.prev = current.prev
            else:
                self._tail = current.prev
            self._size -= 1
            return True
        current = current.next
    return False
```

```
def traverse_backward(self) -> list:
    result = []
    current = self._tail
    while current is not None:
        result.append(current.data)
        current = current.prev
    return result
```

ทดสอบการทำงาน

```
dll = DoublyLinkedList()
print(dll)
dll.append(20)
dll.append(30)
dll.prepend(10)
print(dll)
print(len(dll))

dll.insert_after(20, 25)
print(dll)

print(dll.traverse_backward())

dll.delete(30)
print(dll)
print(dll.traverse_backward())

dll.append(99)
print(dll)
```

None

10 ⇄ 20 ⇄ 30 ⇄ None
3

10 ⇄ 20 ⇄ 25 ⇄ 30 ⇄ None

[30, 25, 20, 10]

10 ⇄ 20 ⇄ 25 ⇄ None
[25, 20, 10]

10 ⇄ 20 ⇄ 25 ⇄ 99 ⇄ None

เปรียบเทียบ Operation ระหว่าง Array, Singly LL, และ Doubly LL

Operation	Dynamic Array	Singly LL	Doubly LL
Access by index	**O(1)**	O(n)	O(n)
Search	O(n)	O(n)	O(n)
Prepend	O(n)	**O(1)**	**O(1)**
Append	O(1)*	O(1)†	**O(1)**
Insert at known node	O(n)	**O(1)**	**O(1)**
Delete head	O(n)	**O(1)**	**O(1)**
Delete tail	**O(1)**	O(n)	**O(1)**
Delete at known node	O(n)	O(n)‡	**O(1)**
Memory layout	Contiguous	Scattered	Scattered
Cache performance	Excellent	Poor	Poor

*amortized † ต้องมี tail pointer ‡ ต้อง traverse หา prev ก่อนเสมอ

ข้อสังเกตที่ 1 — ไม่มีโครงสร้างใดเหนือกว่าอีกโครงสร้างหนึ่งในทุกด้าน

ข้อสังเกตที่ 2 — Doubly LL คือ “ส่วนขยาย” ของ Singly LL ไม่ใช่โครงสร้างที่แยกจากกันโดยสิ้นเชิง

- ทุกแถวที่ Singly LL ทำได้ O(1) นั้น Doubly LL ก็ทำได้ O(1) เช่นกัน
- สิ่งที่ Doubly LL ทำได้ดีขึ้นอย่างชัดเจนมีเพียงสองจุดคือ Delete tail (จาก O(n) กลายเป็น O(1)) และ Delete at known node (จาก O(n) กลายเป็น O(1)) เป็นผลมาจาก prev

ข้อสังเกตที่ 3 — “Insert at known node” คือจุดที่ Linked List เหนือกว่า Array อย่างชัดเจนที่สุด

- ในสถานการณ์จริงจำนวนมาก เราทราบตำแหน่งที่ต้องการแทรกหรือลบข้อมูลอยู่แล้วเสมอ การมีโครงสร้างที่ทำ operation เหล่านี้ได้ในเวลาคงที่จึงมีผลอย่างมากต่อประสิทธิภาพของโปรแกรมเมื่อข้อมูลมีขนาดใหญ่ขึ้น